

Operating Systems - CS(D)

Quiz-4 Solution

1 What is a critical section and how do threads synchronize access to it?

A **critical section** is a part of the program where a thread accesses shared resources such as memory, variables, or files. To avoid data inconsistency due to concurrent access, only one thread must enter the critical section at a time.

Synchronization techniques used:

- `pthread_mutex_lock` and `pthread_mutex_unlock` - Mutual exclusion locks
- Semaphores - Counting mechanism for resource control
- Spinlocks - Busy-waiting locks for short-duration critical sections
- Peterson's or Dekker's Algorithm - Software-based solutions for mutual exclusion

2 What happens if you try to join a detached thread?

Using `pthread_join` on a detached thread leads to an **error (EINVAL)**. A detached thread releases its resources on termination and cannot be joined. Attempting to do so is a programming error and typically returns error code 22 (invalid argument). Once a thread is detached using `pthread_attr_setdetachstate` or `pthread_detach`, it becomes non-joinable and operates independently.

3 Detached Thread for Periodic API Check

This program demonstrates using a detached thread to check updates every 10 minutes while the main application continues running:

```
1 #include <iostream>
2 #include <pthread.h>
3 #include <unistd.h> // for sleep
4
5 using namespace std;
6
7 void* checkUpdates(void*) {
8     while (true) {
9         cout << "Checking for updates..." << endl;
10        sleep(2); // Simulate API call
11        cout << "Update complete." << endl;
12        sleep(600); // Wait 10 minutes (600 seconds)
13    }
14    return nullptr;
15 }
```

```

16
17 int main() {
18     pthread_t updater;
19     pthread_attr_t attr;
20
21     // Initialize and set thread attributes
22     pthread_attr_init(&attr);
23     pthread_attr_setdetachstate(&attr, PTHREAD_CREATE_DETACHED);
24
25     // Create detached thread
26     if (pthread_create(&updater, &attr, checkUpdates, nullptr) != 0) {
27         cerr << "Failed to create thread." << endl;
28     }
29
30     // Clean up thread attributes
31     pthread_attr_destroy(&attr);
32
33     // Main application continues execution
34     for (int i = 0; i < 5; ++i) {
35         cout << "Main app running..." << endl;
36         sleep(3);
37     }
38
39     cout << "Main app exiting." << endl;
40     return 0;
41 }

```

Listing 1: Detached Thread for Periodic API Check

4 Peterson's Algorithm with Entry Counters

Modified Peterson's algorithm to track how many times each process enters its critical section:

Shared Variables

```

1 flag[2] := {false, false} // Indicates process wants to enter critical section
2 turn := 0 // Whose turn is it?
3 count[2] := {0, 0} // Entry counters for each process

```

Algorithm 1 Process 0

```

1: while true do
2:   flag[0] := true
3:   turn := 1
4:   while flag[1] = true and turn = 1 do
    ▷ Busy wait
5:   end while
6:   /* Critical Section */
7:   count[0] := count[0] + 1
8:   flag[0] := false
9:   /* Remainder Section */
10: end while

```

Algorithm 2 Process 1

```

1: while true do
2:   flag[1] := true
3:   turn := 0
4:   while flag[0] = true and turn = 0 do
    ▷ Busy wait
5:   end while
6:   /* Critical Section */
7:   count[1] := count[1] + 1
8:   flag[1] := false
9:   /* Remainder Section */
10: end while

```
